



**SWAMINARAYAN
UNIVERSITY**
THE RESERVOIR OF ALL KNOWLEDGE

Faculty of Engineering (Degree)
Semester – 5th
Subject Name –Computer Networks
Subject Code - BE103404
Faculty Name – Radhika Gokani

Chapter -2
Application Layer

2. Introduction

The **Application Layer** is the topmost layer in both the OSI and TCP/IP models. It serves as the interface between the end-users and the network services, facilitating various network activities.

Functions of the Application Layer

1. **Identifying Communication Partners:** Determines the availability of communication partners for an application with data to transmit.
2. **Resource Availability:** Ensures that sufficient network resources are available for the requested communication.
3. **Synchronizing Communication:** Manages the synchronization of communication between applications.

Services Provided by the Application Layer

- **Email Services:** Facilitates email forwarding and storage.
- **File Transfer, Access, and Management (FTAM):** Allows users to access, retrieve, and manage files on remote computers.
- **Directory Services:** Provides access to global information about various objects and services.
- **Network Virtual Terminal:** Enables users to log on to a remote host by creating a software emulation of a terminal.

Protocols in the Application Layer

Some common protocols operating at this layer include:

- **HTTP/HTTPS:** Used for web browsing.
- **FTP:** Used for file transfers.
- **SMTP:** Used for sending emails.
- **DNS:** Used for resolving domain names to IP addresses.

The Application Layer is crucial as it directly interacts with the end-user and provides the necessary services to facilitate communication over the network.

2.1 Principles of computer applications

The principles of computer applications encompass several key concepts that guide the design, development, and use of applications running on computer networks:

1. Network Application Architectures

- **Client-Server Architecture:** One component acts as a client making requests, and another acts as a server providing services. Common in web applications.
- **Peer-to-Peer (P2P) Architecture:** Each component can act as both a client and a server, communicating directly with each other. Used in file-sharing applications.
- **Three-Tier Architecture:** Divides the application into client, middleware, and database components, improving performance and scalability.
- **Microservices Architecture:** Consists of small, independently deployable components (microservices) that communicate through APIs, offering flexibility and scalability¹.

2. Processes Communicating

- Communication between processes (programs or tasks) on the same or different devices is facilitated by protocols that define rules and formats for data exchange¹.

3. Application-Layer Protocols

- Define rules for data exchange between applications, including formatting, error detection, correction, and security. Examples include HTTP, HTTPS, FTP, SMTP, POP3, and IMAP.

4. User Interface Design

- Focuses on creating intuitive and user-friendly interfaces that enhance user experience and accessibility.

5. Security and Privacy

- Ensures that applications are secure from unauthorized access and data breaches, protecting user data and privacy.

6. Performance and Scalability

- Applications should be designed to perform efficiently under various loads and be scalable to handle increasing amounts of data and users.

7. Maintainability and Flexibility

- Applications should be easy to maintain and update, with a modular design that allows for flexibility and future enhancements.

These principles form the backbone of modern computer applications, ensuring they are efficient, secure, and user-friendly.

2.2 web and HTTP

The Web

The World Wide Web (WWW) is a system of interlinked hypertext documents accessed via the Internet. These documents are written in HTML (HyperText Markup Language) and can include text, images, videos, and other multimedia. Users can navigate between these documents using hyperlinks.

HTTP (Hypertext Transfer Protocol)

HTTP is the protocol used for transferring data on the Web. It defines how messages are formatted and transmitted, and how web servers and browsers should respond to various commands. Here are some key points about HTTP:

1. **Client-Server Model:** HTTP operates on a client-server model. The client (usually a web browser) sends an HTTP request to the server, which then processes the request and sends back an HTTP response.

2. **Request-Response Cycle:** The basic flow involves the client sending a request (e.g., for a web page), and the server responding with the requested resource (e.g., the HTML file of the web page).
3. **HTTP Methods:** There are several methods used in HTTP, including:
 - **GET:** Requests data from a specified resource.
 - **POST:** Submits data to be processed to a specified resource.
 - **PUT:** Updates or creates a resource.
 - **DELETE:** Deletes a specified resource.
4. **Stateless Protocol:** HTTP is stateless, meaning each request from a client to server is independent; the server does not retain any state information between requests.
5. **Versions:** HTTP has evolved over time:
 - **HTTP/1.0:** Introduced in 1996.
 - **HTTP/1.1:** Introduced in 1997, with persistent connections and chunked transfer encoding.
 - **HTTP/2:** Introduced in 2015, with multiplexing and header compression.
 - **HTTP/3:** Based on the QUIC protocol, designed to improve performance.

How It Works

When you type a URL into your browser, here's what happens:

1. **DNS Lookup:** The browser contacts a DNS server to resolve the domain name into an IP address.
2. **TCP Connection:** The browser establishes a TCP connection with the server.
3. **HTTP Request:** The browser sends an HTTP request to the server.
4. **Server Response:** The server processes the request and sends back an HTTP response, which includes the requested resource.
5. **Rendering:** The browser renders the resource (e.g., displays the web page)

2.3 E-mail

E-mail, or electronic mail, is a method of exchanging digital messages over a network, typically the Internet. It allows users to send and receive text, graphics, sounds, and even animated images.

1. **Protocols:** E-mail relies on several protocols within the TCP/IP suite. The Simple Mail Transfer Protocol (SMTP) is used to send messages, while protocols like IMAP (Internet Message Access Protocol) and POP (Post Office Protocol) are used to retrieve messages from a mail server.
2. **E-mail Servers:** These servers handle the sending, receiving, and storage of e-mail messages. When you send an e-mail, it is transferred from your e-mail client to an outgoing mail server using SMTP. The server then forwards the message to the recipient's mail server, where it can be retrieved using IMAP or POP.
3. **Components:** An e-mail message typically includes a header (with information like the sender, recipient, subject, and date) and a body (the main content of the message). Attachments can also be included, allowing users to send files along with their messages.
4. **Security:** E-mail security is crucial, as it involves protecting the content of messages from unauthorized access. This can include encryption, authentication protocols, and secure connections (like SSL/TLS) to ensure that messages are transmitted securely.

2.4 DNS in computer network

The Domain Name System (DNS) is an important component of the internet infrastructure. It acts like the internet's phonebook, translating human-friendly domain names (like `www.example.com`) into numerical IP addresses (like `192.0.2.1`) that computers use to identify each other on the network¹².

How DNS Works

When you type a web address into your browser, the DNS resolution process involves several steps:

1. **DNS Query:** Your computer sends a query to a DNS resolver, usually provided by your Internet Service Provider (ISP).
2. **Recursive Resolver:** The resolver checks its cache to see if it already has the IP address for the domain. If not, it queries other DNS servers.
3. **Root Nameserver:** The resolver first contacts a root nameserver, which directs it to the appropriate Top-Level Domain (TLD) nameserver (e.g., .com, .org).
4. **TLD Nameserver:** The TLD nameserver then directs the resolver to the authoritative nameserver for the specific domain.
5. **Authoritative Nameserver:** This server provides the IP address associated with the domain name.
6. **Response:** The resolver returns the IP address to your computer, which can then access the website.

Components of DNS

- **Domain Names:** Human-readable names like `www.example.com`.
- **IP Addresses:** Numerical addresses like `192.0.2.1`.
- **DNS Servers:** Include recursive resolvers, root nameservers, TLD nameservers, and authoritative nameservers.

Types of DNS Records

- **A Record:** Maps a domain to an IPv4 address.
- **AAAA Record:** Maps a domain to an IPv6 address.
- **CNAME Record:** Alias of one name to another.
- **MX Record:** Mail exchange servers for email routing.
- **TXT Record:** Text information for various purposes, often for verification.

Importance of DNS

- **User Convenience:** Allows users to use easy-to-remember domain names instead of complex IP addresses.
- **Scalability:** Supports the vast number of devices and services on the internet.
- **Redundancy:** Multiple DNS servers ensure reliability and availability.

DNS is fundamental to the functioning of the internet, ensuring that users can easily access websites and services.

2.5 Socket programming with TCP and UDP

Socket programming is a way to enable communication between two nodes on a network. It can be done using two main protocols: TCP (Transmission Control Protocol) and UDP (User Datagram Protocol). Here's a brief overview of both:

TCP Socket Programming

TCP is a connection-oriented protocol, which means a connection is established before data can be sent. It ensures reliable and ordered delivery of data.

Steps for TCP Socket Programming

1. Server Side:
 - Create a socket: Use the `socket()` function.
 - Bind the socket: Associate the socket with an IP address and port using the `bind()` function.
 - Listen for connections: Use the `listen()` function to wait for client connections.
 - Accept a connection: Use the `accept()` function to establish a connection with a client.
 - Send/Receive data: Use `send()` and `recv()` functions to communicate.
 - Close the connection: Use the `close()` function.
2. Client Side:
 - Create a socket: Use the `socket()` function.
 - Connect to the server: Use the `connect()` function to establish a connection.
 - Send/Receive data: Use `send()` and `recv()` functions to communicate.
 - Close the connection: Use the `close()` function.

UDP Socket Programming

UDP is a connectionless protocol, which means data can be sent without establishing a connection. It is faster but does not guarantee reliable or ordered delivery.

Steps for UDP Socket Programming

1. Server Side:
 - Create a socket: Use the `socket()` function.
 - Bind the socket: Associate the socket with an IP address and port using the `bind()` function.
 - Receive data: Use the `recvfrom()` function to receive data.
 - Send data: Use the `sendto()` function to send data.
 - Close the socket: Use the `close()` function.

2. Client Side:

- Create a socket: Use the `socket()` function.
- Send data: Use the `sendto()` function to send data.
- Receive data: Use the `recvfrom()` function to receive data.
- Close the socket: Use the `close()` function.

Key Differences

- Connection: TCP requires a connection to be established, while UDP does not.
- Reliability: TCP ensures reliable delivery, while UDP does not.
- Speed: UDP is generally faster due to the lack of connection overhead.

Socket programming is essential for creating networked applications, such as web servers, chat applications, and more.

TCP socket programming in C, demonstrating both the server and client sides.

Server Side

1. **Create a socket:** Use the `socket()` function.
2. **Bind the socket:** Associate the socket with an IP address and port using the `bind()` function.
3. **Listen for connections:** Use the `listen()` function to wait for client connections.
4. **Accept a connection:** Use the `accept()` function to establish a connection with a client.
5. **Send/Receive data:** Use `send()` and `recv()` functions to communicate.
6. **Close the connection:** Use the `close()` function.

Server Code Example

```
#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>
```

```

#define MAX 80
#define PORT 8080
#define SA struct sockaddr

void func(int connfd) {
    char buff[MAX];
    int n;
    for (;;) {
        bzero(buff, MAX);
        read(connfd, buff, sizeof(buff));
        printf("From client: %s\t To client : ", buff);
        bzero(buff, MAX);
        n = 0;
        while ((buff[n++] = getchar()) != '\n');
        write(connfd, buff, sizeof(buff));
        if (strncmp("exit", buff, 4) == 0) {
            printf("Server Exit...\n");
            break;
        }
    }
}

int main() {
    int sockfd, connfd, len;
    struct sockaddr_in servaddr, cli;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);
    if (sockfd == -1) {
        printf("Socket creation failed...\n");
        exit(0);
    } else {
        printf("Socket successfully created..\n");
    }
    bzero(&servaddr, sizeof(servaddr));

    servaddr.sin_family = AF_INET;

```

```

servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(PORT);

if ((bind(sockfd, (SA*)&servaddr, sizeof(servaddr))) != 0) {
    printf("Socket bind failed...\n");
    exit(0);
} else {
    printf("Socket successfully binded..\n");
}

if ((listen(sockfd, 5)) != 0) {
    printf("Listen failed...\n");
    exit(0);
} else {
    printf("Server listening..\n");
}
len = sizeof(cli);

connfd = accept(sockfd, (SA*)&cli, &len);
if (connfd < 0) {
    printf("Server accept failed...\n");
    exit(0);
} else {
    printf("Server accepted the client...\n");
}

func(connfd);
close(sockfd);
}

```

Client Side

1. **Create a socket:** Use the `socket()` function.
2. **Connect to the server:** Use the `connect()` function to establish a connection.
3. **Send/Receive data:** Use `send()` and `recv()` functions to communicate.
4. **Close the connection:** Use the `close()` function.

Client Code Example

```
#include <stdio.h>
#include <netdb.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <unistd.h>

#define MAX 80
#define PORT 8080
#define SA struct sockaddr

void func(int sockfd) {
    char buff[MAX];
    int n;
    for (;;) {
        bzero(buff, sizeof(buff));
        printf("Enter the string : ");
        n = 0;
        while ((buff[n++] = getchar()) != '\n');
        write(sockfd, buff, sizeof(buff));
        bzero(buff, sizeof(buff));
        read(sockfd, buff, sizeof(buff));
        printf("From Server : %s", buff);
        if ((strncmp(buff, "exit", 4)) == 0) {
            printf("Client Exit...\n");
            break;
        }
    }
}

int main() {
    int sockfd, connfd;
    struct sockaddr_in servaddr, cli;
```

```

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd == -1) {
    printf("Socket creation failed...\n");
    exit(0);
} else {
    printf("Socket successfully created..\n");
}
bzero(&servaddr, sizeof(servaddr));

servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
servaddr.sin_port = htons(PORT);

if (connect(sockfd, (SA*)&servaddr, sizeof(servaddr)) != 0) {
    printf("Connection with the server failed...\n");
    exit(0);
} else {
    printf("Connected to the server..\n");
}

func(sockfd);
close(sockfd);
}

```

This example demonstrates a simple chat application where the client sends a message to the server, and the server echoes it back. The connection is terminated when the client sends the message "exit".